



Universidad Agraria Del Ecuador

Carrera: Computación En Línea

TAREA INDIVIDUAL: Merge Sort y Quick Sort

Asignatura

Lenguaje De Programación 2

Docente

Ing. Rodrigo Pazmiño

Curso

3S "D"

Estudiante

Melany Del Rocio Quintana Bedor

Fecha

Miércoles, 24 junio de 2026



Ejercicios con Merge Sort

1. Ordenamiento de edades

Dada una lista de edades de estudiantes:

edades = [21, 18, 25, 19, 22, 17, 20]

Ordene las edades de menor a mayor utilizando **Merge Sort**.

```
1 # Función Merge Sort
2 def merge_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Dividir la lista en dos mitades
8     mitad = len(lista) // 2
9     izquierda = merge_sort(lista[:mitad])
10    derecha = merge_sort(lista[mitad:])
11
12    # Unir las dos mitades ordenadas
13    return merge(izquierda, derecha)
14
15
16 # Función para combinar las listas ordenadas
17 def merge(izquierda, derecha):
18     resultado = []
19     i = 0
20     j = 0
21
22     while i < len(izquierda) and j < len(derecha):
23         if izquierda[i] < derecha[j]:
24             resultado.append(izquierda[i])
25             i += 1
26         else:
27             resultado.append(derecha[j])
28             j += 1
29
30     # Agregar los elementos restantes
31     resultado.extend(izquierda[i:])
32     resultado.extend(derecha[j:])
33
34     return resultado
35
36
37 # Lista de edades
38 edades = [21, 18, 25, 19, 22, 17, 20]
39
40 print("Lista original:")
41 print(edades)
42
43 # Ordenar con Merge Sort
44 edades_ordenadas = merge_sort(edades)
45
46 print("\nLista ordenada de menor a mayor:")
47 print(edades_ordenadas)
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PU
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Py
Lista original:
[21, 18, 25, 19, 22, 17, 20]

Lista ordenada de menor a mayor:
[17, 18, 19, 20, 21, 22, 25]
PS C:\Users\HP>
```

Pregunta crítica:

¿Por qué Merge Sort divide la lista antes de ordenarla?

Merge Sort divide la lista en partes más pequeñas porque es más fácil ordenar listas cortas que una lista grande. Después de ordenar cada parte por separado, las une de forma ordenada para obtener la lista completa. Esta estrategia de "divide y vencerás" hace que el algoritmo sea eficiente y rápido incluso cuando se trabaja con una gran cantidad de datos.



3. Ordenamiento de precios

Dada una lista de precios de productos:

precios = [15.50, 8.99, 20.00, 5.75, 12.30]

Ordene los precios de menor a mayor aplicando **Merge Sort**.

```
1  # Función Merge Sort
2  def merge_sort(lista):
3      # Caso base
4      if len(lista) <= 1:
5          return lista
6
7      # Dividir la lista en dos mitades
8      mitad = len(lista) // 2
9      izquierda = merge_sort(lista[:mitad])
10     derecha = merge_sort(lista[mitad:])
11
12     # Unir las dos mitades ordenadas
13     return merge(izquierda, derecha)
14
15
16 # Función para combinar las listas ordenadas
17 def merge(izquierda, derecha):
18     resultado = []
19     i = 0
20     j = 0
21
22     while i < len(izquierda) and j < len(derecha):
23         if izquierda[i] < derecha[j]:
24             resultado.append(izquierda[i])
25             i += 1
26         else:
27             resultado.append(derecha[j])
28             j += 1
29
30     # Agregar los elementos restantes
31     resultado.extend(izquierda[i:])
32     resultado.extend(derecha[j:])
33
34     return resultado
35
36
37 # Lista de precios
38 precios = [15.50, 8.99, 20.00, 5.75, 12.30]
39
40 print("Lista original:")
41 print(precios)
42
43 # Ordenar con Merge Sort
44 precios_ordenados = merge_sort(precios)
45
46 print("\nLista ordenada de menor a mayor:")
47 print(precios_ordenados)
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

PS C:\Users\HP> & C:/Users/HP/AppData/Local/Program
Lista original:
[15.5, 8.99, 20.0, 5.75, 12.3]

Lista ordenada de menor a mayor:
[5.75, 8.99, 12.3, 15.5, 20.0]
PS C:\Users\HP>
```

Pregunta crítica:

¿Por qué Merge Sort necesita combinar nuevamente las sublistas?

Merge Sort combina nuevamente las sublistas porque, después de dividir la lista original, cada parte se ordena por separado. Al unir las de forma ordenada, se obtiene una única lista completamente ordenada. Este proceso garantiza que todos los elementos queden en el orden correcto de menor a mayor.



4. Ordenamiento de nombres

Dada una lista de nombres:

nombres = ["Carlos", "Ana", "Luis", "Beatriz", "Daniel"]

Ordene los nombres alfabéticamente usando **Merge Sort**.

```
1 # Función Merge Sort
2 def merge_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Dividir la lista en dos mitades
8     mitad = len(lista) // 2
9     izquierda = merge_sort(lista[:mitad])
10    derecha = merge_sort(lista[mitad:])
11
12    # Unir las dos mitades ordenadas
13    return merge(izquierda, derecha)
14
15
16 # Función para combinar las listas ordenadas
17 def merge(izquierda, derecha):
18     resultado = []
19     i = 0
20     j = 0
21
22     while i < len(izquierda) and j < len(derecha):
23         if izquierda[i] < derecha[j]:
24             resultado.append(izquierda[i])
25             i += 1
26         else:
27             resultado.append(derecha[j])
28             j += 1
29
30     # Agregar los elementos restantes
31     resultado.extend(izquierda[i:])
32     resultado.extend(derecha[j:])
33
34     return resultado
35
36
37 # Lista de nombres
38 nombres = ["Carlos", "Ana", "Luis", "Beatriz", "Daniel"]
39
40 print("Lista original:")
41 print(nombres)
42
43 # Ordenar con Merge Sort
44 nombres_ordenados = merge_sort(nombres)
45
46 print("\nLista ordenada alfabéticamente:")
47 print(nombres_ordenados)
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs
Lista original:
['Carlos', 'Ana', 'Luis', 'Beatriz', 'Daniel']

Lista ordenada alfabéticamente:
['Ana', 'Beatriz', 'Carlos', 'Daniel', 'Luis']
PS C:\Users\HP>
```

Pregunta crítica:

¿Merge Sort puede aplicarse también a datos de tipo texto? Explique.

Sí. Merge Sort puede aplicarse a datos de tipo texto porque compara los elementos utilizando el orden alfabético (orden lexicográfico). El algoritmo funciona con cualquier tipo de dato que pueda compararse, como números, cadenas de texto, fechas o caracteres, por lo que también es adecuado para ordenar nombres alfabéticamente.



5. Ordenamiento descendente de ventas

Dada una lista de ventas mensuales:

ventas = [250, 100, 500, 300, 150, 400]

Ordene las ventas de mayor a menor utilizando **Merge Sort**.

```
1 # Función Merge Sort
2 def merge_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Dividir la lista en dos mitades
8     mitad = len(lista) // 2
9     izquierda = merge_sort(lista[:mitad])
10    derecha = merge_sort(lista[mitad:])
11
12    # Unir las dos mitades ordenadas
13    return merge(izquierda, derecha)
14
15
16 # Función para combinar las listas en orden descendente
17 def merge(izquierda, derecha):
18     resultado = []
19     i = 0
20     j = 0
21
22     while i < len(izquierda) and j < len(derecha):
23         # Comparación para ordenar de mayor a menor
24         if izquierda[i] > derecha[j]:
25             resultado.append(izquierda[i])
26             i += 1
27         else:
28             resultado.append(derecha[j])
29             j += 1
30
31     # Agregar los elementos restantes
32     resultado.extend(izquierda[i:])
33     resultado.extend(derecha[j:])
34
35     return resultado
36
37
38 # Lista de ventas
39 ventas = [250, 100, 500, 300, 150, 400]
40
41 print("Lista original:")
42 print(ventas)
43
44 # Ordenar con Merge Sort (descendente)
45 ventas_ordenadas = merge_sort(ventas)
46
47 print("\nLista ordenada de mayor a menor:")
48 print(ventas_ordenadas)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs
Lista original:
[250, 100, 500, 300, 150, 400]

Lista ordenada de mayor a menor:
[500, 400, 300, 250, 150, 100]
PS C:\Users\HP>
```

Pregunta crítica:

¿Qué cambio debe realizarse en la comparación para ordenar de forma descendente?

Para ordenar de forma descendente, se debe cambiar la condición de comparación en la función merge. En lugar de usar `izquierda[i] < derecha[j]`, se utiliza `izquierda[i] > derecha[j]`. De esta manera, primero se agrega el elemento de mayor valor, obteniendo la lista ordenada de mayor a menor.



Ejercicios con Quick Sort

6. Ordenamiento de números enteros

Dada la siguiente lista:

numeros = [34, 12, 5, 66, 23, 9, 40]

Ordene los números de menor a mayor utilizando **Quick Sort**

```
1 # Función Quick Sort
2 def quick_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Seleccionar el pivote
8     pivote = lista[len(lista) // 2]
9
10    # Dividir la lista
11    menores = [x for x in lista if x < pivote]
12    iguales = [x for x in lista if x == pivote]
13    mayores = [x for x in lista if x > pivote]
14
15    # Ordenar recursivamente
16    return quick_sort(menores) + iguales + quick_sort(mayores)
17
18
19 # Lista de números
20 numeros = [34, 12, 5, 66, 23, 9, 40]
21
22 print("Lista original:")
23 print(numeros)
24
25 # Ordenar con Quick Sort
26 numeros_ordenados = quick_sort(numeros)
27
28 print("\nLista ordenada de menor a mayor:")
29 print(numeros_ordenados)
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PU
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python39-64/Python.exe
Lista original:
[34, 12, 5, 66, 23, 9, 40]

Lista ordenada de menor a mayor:
[5, 9, 12, 23, 34, 40, 66]
PS C:\Users\HP>
```

Pregunta crítica:

¿Cuál es la función del pivote en Quick Sort?

El pivote es el elemento que se utiliza para dividir la lista en dos partes: una con los elementos menores que el pivote y otra con los elementos mayores. Después, cada parte se ordena de forma independiente. La correcta elección del pivote influye en la eficiencia del algoritmo, ya que puede reducir el número de comparaciones y mejorar el rendimiento del ordenamiento.



7. Ordenamiento de estaturas

Dada una lista de estaturas:

estaturas = [1.70, 1.55, 1.80, 1.62, 1.68]

Ordene las estaturas de menor a mayor usando **Quick Sort**.

```
1 # Función Quick Sort
2 def quick_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Seleccionar el pivote
8     pivote = lista[len(lista) // 2]
9
10    # Dividir la lista
11    menores = [x for x in lista if x < pivote]
12    iguales = [x for x in lista if x == pivote]
13    mayores = [x for x in lista if x > pivote]
14
15    # Ordenar recursivamente
16    return quick_sort(menores) + iguales + quick_sort(mayores)
17
18
19 # Lista de estaturas
20 estaturas = [1.70, 1.55, 1.80, 1.62, 1.68]
21
22 print("Lista original:")
23 print(estaturas)
24
25 # Ordenar con Quick Sort
26 estaturas_ordenadas = quick_sort(estaturas)
27
28 print("\nLista ordenada de menor a mayor:")
29 print(estaturas_ordenadas)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python38-32/Python.exe C:\Users\HP\Documents\Programacion\27\Ordenamiento\7_Ordenamiento_de_estaturas.py
Lista original:
[1.7, 1.55, 1.8, 1.62, 1.68]

Lista ordenada de menor a mayor:
[1.55, 1.62, 1.68, 1.7, 1.8]
PS C:\Users\HP>
```

Pregunta crítica:

¿Qué sucede si se elige un pivote poco adecuado?

Si se elige un pivote poco adecuado, la lista puede dividirse de forma muy desequilibrada, haciendo que Quick Sort necesite más comparaciones y más llamadas recursivas. Esto disminuye su rendimiento y, en el peor de los casos, su complejidad puede llegar a $O(n^2)$ en lugar de $O(n \log n)$.



8. Ordenamiento de cantidades de productos

Dada una lista con cantidades en inventario:

inventario = [50, 10, 80, 30, 5, 100, 60]

Ordene las cantidades de menor a mayor utilizando **Quick Sort**.

```
1 # Función Quick Sort
2 def quick_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Seleccionar el pivote
8     pivote = lista[len(lista) // 2]
9
10    # Dividir la lista
11    menores = [x for x in lista if x < pivote]
12    iguales = [x for x in lista if x == pivote]
13    mayores = [x for x in lista if x > pivote]
14
15    # Ordenar recursivamente
16    return quick_sort(menores) + iguales + quick_sort(mayores)
17
18
19 # Lista de cantidades en inventario
20 inventario = [50, 10, 80, 30, 5, 100, 60]
21
22 print("Lista original:")
23 print(inventario)
24
25 # Ordenar con Quick Sort
26 inventario_ordenado = quick_sort(inventario)
27
28 print("\nLista ordenada de menor a mayor:")
29 print(inventario_ordenado)
```

```
22 print("Lista original:")
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/
```

```
Lista original:
```

```
[50, 10, 80, 30, 5, 100, 60]
```

```
Lista ordenada de menor a mayor:
```

```
[5, 10, 30, 50, 60, 80, 100]
```

```
PS C:\Users\HP>
```

Pregunta crítica:

¿Por qué Quick Sort suele ser rápido en la práctica?

Quick Sort suele ser rápido porque divide la lista en partes más pequeñas y las ordena de forma recursiva. Cuando el pivote se elige adecuadamente, las divisiones son equilibradas, lo que reduce el número de comparaciones y permite alcanzar un rendimiento promedio de $O(n \log n)$. Además, requiere poca memoria adicional, por lo que es uno de los algoritmos de ordenamiento más utilizados.



9. Ordenamiento descendente de puntajes

Dada una lista de puntajes:

puntajes = [450, 300, 700, 600, 250, 900]

Ordene los puntajes de mayor a menor aplicando **Quick Sort**.

```
1 # Función Quick Sort (orden descendente)
2 def quick_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Seleccionar el pivote
8     pivote = lista[len(lista) // 2]
9
10    # Dividir la lista para orden descendente
11    mayores = [x for x in lista if x > pivote]
12    iguales = [x for x in lista if x == pivote]
13    menores = [x for x in lista if x < pivote]
14
15    # Ordenar recursivamente
16    return quick_sort(mayores) + iguales + quick_sort(menores)
17
18
19 # Lista de puntajes
20 puntajes = [450, 300, 700, 600, 250, 900]
21
22 print("Lista original:")
23 print(puntajes)
24
25 # Ordenar con Quick Sort (descendente)
26 puntajes_ordenados = quick_sort(puntajes)
27
28 print("\nLista ordenada de mayor a menor:")
29 print(puntajes_ordenados)
```

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUEB...

PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Pyt...
Lista original:
[450, 300, 700, 600, 250, 900]

Lista ordenada de mayor a menor:
[900, 700, 600, 450, 300, 250]
PS C:\Users\HP>
```

Pregunta crítica:

¿Cómo se modifica Quick Sort para ordenar de forma descendente?

Para ordenar de forma descendente, se cambia la forma en que se dividen los elementos con respecto al pivote. En lugar de colocar primero los elementos menores, se colocan primero los mayores, luego los iguales y finalmente los menores. Así, al unir las listas, el resultado queda ordenado de mayor a menor.



10. Ordenamiento de tiempos de carrera

Dada una lista de tiempos en segundos:

tiempos = [12.5, 10.8, 14.2, 11.9, 13.0]

Ordene los tiempos de menor a mayor utilizando **Quick Sort**.

```
1 # Función Quick Sort
2 def quick_sort(lista):
3     # Caso base
4     if len(lista) <= 1:
5         return lista
6
7     # Seleccionar el pivote
8     pivote = lista[len(lista) // 2]
9
10    # Dividir la lista
11    menores = [x for x in lista if x < pivote]
12    iguales = [x for x in lista if x == pivote]
13    mayores = [x for x in lista if x > pivote]
14
15    # Ordenar recursivamente
16    return quick_sort(menores) + iguales + quick_sort(mayores)
17
18
19 # Lista de tiempos
20 tiempos = [12.5, 10.8, 14.2, 11.9, 13.0]
21
22 print("Lista original:")
23 print(tiempos)
24
25 # Ordenar con Quick Sort
26 tiempos_ordenados = quick_sort(tiempos)
27
28 print("\nLista ordenada de menor a mayor:")
29 print(tiempos_ordenados)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe
Lista original:
[12.5, 10.8, 14.2, 11.9, 13.0]

Lista ordenada de menor a mayor:
[10.8, 11.9, 12.5, 13.0, 14.2]
PS C:\Users\HP>
```

Pregunta crítica:

¿Por qué en este caso el menor tiempo debe quedar primero?

Porque en una carrera, el menor tiempo representa el mejor desempeño. Ordenar los tiempos de menor a mayor permite identificar fácilmente al ganador y establecer la clasificación de los participantes, desde el más rápido hasta el más lento.



Recomendación final

Compare los resultados de Merge Sort y Quick Sort. Ambos son algoritmos avanzados, pero tienen diferencias importantes: Merge Sort es más estable y divide la lista en partes iguales, mientras que Quick Sort depende de la elección del pivote y suele ser muy eficiente en la práctica.

Merge Sort y Quick Sort son algoritmos de ordenamiento eficientes con una complejidad promedio de $O(n \log n)$, pero presentan diferencias importantes. **Merge Sort** divide la lista en partes iguales, mantiene el orden de los elementos iguales (es un algoritmo estable) y ofrece un rendimiento constante, aunque requiere memoria adicional para combinar las sublistas. Por otro lado, **Quick Sort** organiza los datos alrededor de un pivote, utiliza menos memoria y suele ser más rápido en la práctica. Sin embargo, su rendimiento depende de la elección del pivote, ya que una mala selección puede hacer que su complejidad aumente hasta $O(n^2)$ en el peor caso.

Análisis:

Merge Sort es más adecuado cuando se necesita un algoritmo estable y con un rendimiento constante, especialmente para listas grandes o cuando es importante mantener el orden de elementos iguales. Quick Sort suele ser más rápido en la práctica porque requiere menos memoria y tiene un excelente rendimiento promedio. Sin embargo, depende de la elección del pivote. Por ello, ambos algoritmos son eficientes, pero la elección depende del tipo de problema y de las características de los datos.

Conclusión:

En este trabajo se comprobó que ambos algoritmos permiten ordenar correctamente diferentes tipos de datos, como números, nombres, precios y tiempos. La elección entre Merge Sort y Quick Sort dependerá de las necesidades del problema, considerando aspectos como la estabilidad, el uso de memoria y la velocidad de ejecución.